AMRITA VISHWA VIDYAPEETHAM
Amrita School of Computing, Amritapuri Campus
Department of Computer Science and Engineering

ACADEMIC CASE STUDY REPORT · 23CSE451 GRAPH MINING

RouteWise: Real-Road Route Optimization and Spatial Taxi Demand Analytics Using Graph Mining

Empirical Evaluation of Directed OpenStreetMap Road Topology, Geodesic A Routing, Sampled Edge Betweenness Resilience, and Spatial Gravity Demand Communities in Karunagappally, Kerala*

	23CSE451 - Graph Mining (L-T-P-C: 3-0-0-3)
Academic Year & Semester	Academic Year: 2026-2027 | Semester VII (B.Tech CSE)
Student Name	Arjun A.
Register Number	AM.SC.U4CSE23011
Evaluation Component	Course Case Study & Empirical Implementation (Full Marks)
Date of Submission	October 29, 2026

1. Abstract

This case study presents RouteWise, an interactive dual-graph mining and geospatial optimization system focused on road connectivity and taxi mobility patterns across the urban agglomeration of Karunagappally, Kerala. We model the physical transportation infrastructure as a real directed OpenStreetMap (OSM) MultiDiGraph containing 9,634 vertices and 24,899 directed road segments, preserving one-way restrictions, turn angles, and parallel carriageway geometries. For routing, we implement and benchmark NetworkX Dijkstra and geodesic A* using an admissible Haversine straight-line distance heuristic, verifying path length parity against the Open Source Routing Machine (OSRM) driving API. Structural network resilience is evaluated through sampled edge betweenness centrality (k=500 pivots via Brandes' algorithm) and systematic edge-closure simulations across 665 road segments, identifying single-point-of-failure bottlenecks such as coastal canal bridges where single edge failures induce up to +105.56% (+8.92 km) detours. In parallel, a spatial gravity interaction model incorporating landmark demographic weights, transit hubs, and calibrated distance-decay (γ=1.3) generates a deterministic synthetic demand graph of 5,000 trips across 12 regional landmarks. Graph mining techniques—including weighted PageRank, nodal betweenness centrality, and Newman-Girvan modularity optimization via the Louvain algorithm across resolutions 0.5 to 1.5—uncover consistent community partitions reflecting regional commercial cores versus coastal transit links. The study establishes rigorous empirical separation between physical road networks and simulated mobility demand, presenting full reproducibility metrics and concrete algorithmic complexity profiles.

> **Primary Theoretical Insight**
> *Core Graph Mining Takeaway: Physical road topology and mobility demand operate under fundamentally distinct structural paradigms. While road graphs exhibit planar sparsity, low maximum degree (k_max=6), and vulnerability localized to physical bridges, origin-destination demand networks form dense, weighted multi-hub cliques governed by distance decay and land-use attractiveness.*

2. Introduction

Urban transportation networks represent the quintessential domain for applied graph mining. Modern municipal transit systems grapple with escalating vehicular congestion, infrastructure bottlenecks, and dynamically shifting commuter demand. Karunagappally, an expanding commercial municipality situated along National Highway 66 (NH 66) in the Kollam district of Kerala, presents a compelling real-world laboratory. The region is characterized by complex topological constraints: a high-capacity north-south arterial highway, dense commercial ribbon development, coastal waterways and tidal inlets (such as the TS Canal and National Waterway 3), intermodal transport hubs (Karunagappally Railway Station and KSRTC Bus Depot), and educational clusters including Amrita Vishwa Vidyapeetham at Amritapuri.

Graph mining offers mathematical abstractions capable of transforming unstructured spatial geography into rigorous computational models. Specifically, road infrastructure can be modeled as a spatial directed graph, where intersections and terminal points serve as vertices, while roadway

segments represent directed edges weighted by physical length or travel impedance. In contrast, passenger travel demand forms an origin-destination (OD) spatial interaction graph, where vertices represent prominent regional landmarks and weighted edges denote aggregate trip frequencies.

A central flaw in many transportation data projects is the conflation of physical roadway infrastructure with simulated trip distributions, leading to deceptive metrics and ungrounded algorithmic conclusions. RouteWise is specifically designed to enforce rigorous epistemological and methodological separation between these two distinct graph layers:

- **Real Physical Road Graph:** Constructed from verified OpenStreetMap (OSM) vector geometry, maintaining exact geographic coordinates, one-way directional edges, and parallel roadway carriageways for physical shortest-path navigation and vulnerability analysis.
- **Synthetic Taxi Demand Graph:** Generated through a spatial interaction Gravity Model grounded in landmark demographic generation weights, land-use attraction scores, and calibrated distance-decay functions, utilized exclusively for community detection and mobility pattern discovery.

3. Related Work

The graph-theoretic analysis of vehicular networks and urban mobility builds upon several landmark contributions spanning classical path algorithms, complex network analysis, and transportation resilience modeling.

Spatial Road Modeling & OSM Extraction: Boeing (2017) [1] formalized the programmatic retrieval and topological structuring of urban street networks from OpenStreetMap using OSMnx, demonstrating that planar MultiDiGraph representations correctly preserve complex intersection geometries and one-way constraints compared to simplified planar abstractions. Luxen and Vetter (2011) [2] introduced Contraction Hierarchies within the Open Source Routing Machine (OSRM), demonstrating that real-time continental routing requires multi-level graph preprocessing to achieve microsecond-scale query latencies on drivable graphs.

Shortest Path and Heuristic Search: Dijkstra (1959) [3] established the foundational single-source shortest path algorithm running in O((V + E) log V) with binary or Fibonacci heaps. Hart, Nilsson, and Raphael (1968) [4] generalized this into the A* algorithm by integrating heuristic estimators. In geospatial road networks, straight-line Euclidean or Great-Circle (Haversine) distances provide strictly admissible and consistent heuristics, ensuring optimal path discovery while pruning unnecessary graph traversals.

Centrality and Network Resilience: Brandes (2001) [5] revolutionized shortest-path betweenness centrality by developing an exact algorithm running in O(V · E) time for unweighted graphs and O(V · E + V^2 log V) for weighted graphs. In large-scale metropolitan road systems, computing exact betweenness over tens of thousands of vertices becomes computationally prohibitive; Brandes and Pich (2007) and Geisberger et al. established pivot-based stochastic sampling methods that provide provable approximation bounds. Transportation resilience researchers (Jenelius, Petersen, and Mattsson, 2006 [6]; Mattsson and Jenelius, 2015 [7]) formalized critical link identification by measuring network-wide detour ratios under single-edge and multi-edge topological removals, demonstrating that peripheral bridge structures exhibit catastrophic vulnerability compared to grid-like urban cores.

Community Detection on Mobility Networks: Blondel, Guillaume, Lambiotte, and Lefebvre (2008) [8] introduced the Louvain method, a multi-phase greedy optimization algorithm maximizing Newman-Girvan modularity (Newman, 2006 [9]). In transportation analytics, Castro et al. (2013) [10] and Zheng et al. (2014) [11] applied community detection to taxi origin-destination callsets, discovering that mobility communities align with functional socioeconomic zones rather than arbitrary administrative boundaries. Hagberg, Swart, and S Chult (2008) [12] implemented these scalable data structures within the NetworkX framework, enabling reproducible empirical graph science.

4. Problem Statement & Objectives

Problem Statement: Given a municipal region bounded by geographic coordinates B = (N, S, E, W), we seek to: (1) reconstruct the complete physical directed road graph G_road = (V_road, E_road) from OpenStreetMap without loss of multigraph lane geometry; (2) solve optimal routing across regional transit corridors using length-weighted Dijkstra and geodesic A*; (3) quantify structural network vulnerability through sampled edge betweenness centrality and systematic road closure simulations G_road \ {e}; and (4) model and partition spatial taxi mobility demand G_demand = (V_demand, E_demand) into functional clusters using Newman-Girvan modularity optimization across varying resolution scales.

The specific objectives of this study are:

- Construct and validate an offline-cached, directed OpenStreetMap MultiDiGraph for Karunagappally with zero synthetic road artifacts.
- Implement a rigorous Coordinate Snapping mechanism mapping arbitrary inputs to the Largest Strongly Connected Component (SCC), eliminating dead-end entrapment.
- Design a deterministic spatial interaction Gravity Model calibrated for Karunagappally landmarks, replacing ungrounded uniform noise generators.
- Benchmark Dijkstra vs. A* runtime efficiency across 30 repeated runs and compare spatial fidelity against the OSRM public routing engine.
- Systematically evaluate 665 directed road closures along evaluated shortest paths to pinpoint critical topological bottlenecks and measure detour inflation.
- Perform multi-seed Louvain community detection across resolution scales γ in {0.5, 1.0, 1.5} and report stability and modularity scores.

5. Data and Reproducibility

Scientific integrity requires that every parameter, coordinate, and random seed be documented to enable independent reproduction. RouteWise utilizes two distinct datasets described below.

OpenStreetMap Road Graph: Extracted via OSMnx within bounding box North: 9.16°N, South: 8.96°N, East: 76.66°E, West: 76.46°E (area: 22.2 km × 22.0 km ≈ 488 km²). Filter: network_type='drive', simplify=True. Captured and cached to data/karunagappally_drive.graphml on October 29, 2026. The graph contains 9,634 nodes and 24,899 directed edges.

Table 1: Study Area Landmark Coordinates, Demographic Weights, and Roles

Landmark Name	Latitude (°N)	Longitude (°E)	Transit / Regional Role	Pop. Weight (P_i)	Attr. Weight (A_j)
Karunagappally	9.0544	76.5353	Central Municipal Business District	100	100
Railway Station	9.062	76.5401	Southern Railway Intermodal Transit Hub	80	85
KSRTC Bus Stand	9.0531	76.5365	State Road Transport Regional Bus Depot	90	90
Oachira	9.1306	76.5042	Northern Temple Town & Border Junction	45	50
Chavara	8.9967	76.527	Southern Industrial Belt (KMML/IRE)	50	45
Sasthamkotta	9.0436	76.6264	Inland Freshwater Lake & Tourism Hub	40	40
Mynagappally	9.0482	76.5681	Agricultural & Suburban Commuter Village	35	30

Landmark Name	Latitude (°N)	Longitude (°E)	Transit / Regional Role	Pop. Weight (P_i)	Attr. Weight (A_j)
Thazhava	9.0831	76.5583	North-Eastern Residential Township	35	30
Clappana	9.1124	76.4982	North-Western Coastal Fishing Belt	25	25
Vallikavu	9.0898	76.4897	National Waterway 3 Bridge Landing	30	35
Amritapuri	9.0939	76.4918	University Campus & International Ashram	65	70
Puthiyakavu	9.0712	76.5321	Northern Municipal Commercial Junction	45	45

Deterministic Synthetic Demand Model: Real taxi telemetry in Karunagappally is not publicly available from commercial dispatchers. Rather than injecting uniform random noise, we generate N=5,000 trips using a spatial interaction Gravity Model (Zipf, 1946; Wilson, 1971):

Gravity Model Formulation for Synthetic Mobility Generation

```
T_{ij} = k · \frac{P_i^{\alpha} · A_j^{\beta}}{d_{ij}^{\gamma}}
Where:
  - P_i = Origin landmark population/trip-generation weight
  - A_j = Destination landmark economic/transit attraction weight
  - d_{ij} = Verified network shortest-path distance (km) on the real OSM graph
  - alpha = 1.0, beta = 1.0, gamma = 1.3 (calibrated distance decay exponent)
  - k = Normalizing factor ensuring sum(T_{ij}) = 5,000 trips (Seed = 42)
  - Temporal sampling: Peak morning (08:00-10:00) and evening (16:30-19:30) weights
```

Academic Taxi Fare Estimation Formula: Calibrated in accordance with the Kerala Motor Vehicles Department (MVD) fare schedule for auto-rickshaws and regional taxis:

Kerala MVD Grounded Academic Fare Equation

```
Fare(INR) = FARE_BASE + (d_km · FARE_PER_KM) + (t_min · FARE_PER_MIN)
Parameters:
  - FARE_BASE = 45.0 INR (flag drop covering initial 1.5 km)
  - FARE_PER_KM = 18.0 INR / km
  - FARE_PER_MIN = 1.5 INR / min (waiting / congestion transit rate)
  - Evaluated uniformly across Dijkstra routes (at 28 km/h baseline speed) and OSRM
candidates.
```

6. Graph Modeling

RouteWise formalizes the transportation environment into two cleanly separated mathematical graphs:

Physical Road Graph (G_road = (V_road, E_road)): Formatted as a directed MultiDiGraph. Vertices v in V_road represent physical intersections, highway ramps, and dead-ends with geographic attributes (lat, lon, x, y, elevation). Directed edges e = (u, v, k) represent one-way or two-way vehicular road segments, where k is the multigraph key distinguishing parallel lanes or divided highway carriageways. The edge weight w(e) is the physical road segment length in meters. Parallel edge resolution is strictly defined as:

Parallel Edge Weight Resolution

```
w(u, v) = min_{k} { length(u, v, k) }
Guarantees that when multiple parallel links exist between vertices u and v, Dijkstra
selects the exact minimal traversal length.
```

Strong Connectivity & Snapping Protocol: In directed geographic networks, snapping coordinates to the nearest arbitrary node often traps routes in one-way dead-ends. RouteWise resolves this by partitioning G_road into strongly connected components (SCCs). The graph comprises 1 Weakly Connected Component (WCC) and 2 Strongly Connected Components: the giant SCC contains 9,633 nodes (99.99%), with exactly 1 sink node disconnected in the reverse direction. Coordinate snapping is constrained strictly to the giant SCC:

SCC-Constrained Nearest Node Snapping

```
v_snap = argmin_{v in V_{SCC}} { (x_v - lon)^2 + (y_v - lat)^2 }
Guarantees that every snapped origin and destination is mutually reachable across
directed edges.
```

Synthetic Demand Graph (G_demand = (V_demand, E_demand)): A directed, weighted landmark interaction network where V_demand represents the 12 named regional landmarks, and directed edges e = (u, v) carry a weight W_uv equal to the total simulated trip frequency from landmark u to landmark v.

7. Methods and Algorithms

This section details the algorithmic procedures implemented across the RouteWise pipeline.

Algorithm 1: Shortest-Path Computation on Directed MultiDiGraph

```
Algorithm 1: Length-Weighted MultiDiGraph Dijkstra
Input: Directed MultiDiGraph G = (V, E), source node s, target node t, weight attribute
'length'
Output: Optimal vertex path P = [s, v_1, ..., t], distance dist[t]
1: Initialize dist[v] = infinity for all v in V; dist[s] = 0; pred[v] = None
2: Priority queue Q = MinHeap([(0, s)])
3: while Q is not empty do
4:     (d, u) = ExtractMin(Q)
5:     if d > dist[u] then continue
6:     if u == t then break
7:     for each out-neighbor v of u do
8:         w_uv = min_{k} { G.get_edge_data(u, v, k)['length'] }
9:         if dist[u] + w_uv < dist[v] then
10:            dist[v] = dist[u] + w_uv
11:            pred[v] = u
12:            InsertOrDecreaseKey(Q, (dist[v], v))
13: Reconstruct path P from pred back to s; return (P, dist[t])
```

Algorithm 2: Geodesic A* Search with Great-Circle Heuristic

```
Algorithm 2: Geodesic A* with Admissible Haversine Distance Heuristic
Input: Graph G = (V, E), source s, target t
Heuristic h(u, t): Straight-line great-circle distance (meters):
  delta_phi = rad(lat_t - lat_u); delta_lambda = rad(lon_t - lon_u)
  a = sin^2(delta_phi/2) + cos(rad(lat_u)) * cos(rad(lat_t)) * sin^2(delta_lambda/2)
  h(u, t) = 2 * R_earth * asin(sqrt(a))  [Admissible because h(u, t) <= true road
distance]
1: g_score[v] = infinity for all v in V; g_score[s] = 0
2: f_score[v] = infinity; f_score[s] = h(s, t)
3: OpenSet = MinHeap([(f_score[s], s)])
4: while OpenSet is not empty do
5:     (f, u) = ExtractMin(OpenSet)
6:     if u == t then return ReconstructPath(pred, t)
7:     for each out-neighbor v of u do
8:         tentative_g = g_score[u] + min_k { G[u][v][k]['length'] }
9:         if tentative_g < g_score[v] then
10:            pred[v] = u; g_score[v] = tentative_g
11:            f_score[v] = tentative_g + h(v, t)
12:            Push(OpenSet, (f_score[v], v))
```

Algorithm 3: Sampled Edge Betweenness Centrality via Pivot Sampling

```
Algorithm 3: Sampled Edge Betweenness Centrality (Brandes Pivot Sampling)
Input: Directed Graph G = (V, E), sample pivot count k = 500, random seed = 42
Output: Edge betweenness score C_B(e) for all e in E
1: Initialize C_B(e) = 0 for all e in E
2: Randomly sample k distinct pivot source nodes S_p subset of V using seed = 42
3: for each pivot s in S_p do
4:     Run single-source shortest path from s (Dijkstra) tracking path counts sigma[v]
and predecessors P[v]
5:     Initialize nodal dependencies delta[v] = 0 for all v in V
6:     Stack S contains vertices ordered by descending distance from s
7:     while S is not empty do
```

```
8:         Pop w from S
9:         for each predecessor v in P[w] do
10:            c = (sigma[v] / sigma[w]) * (1 + delta[w])
11:            C_B(v, w) = C_B(v, w) + c
12:            delta[v] = delta[v] + c
13: Normalize C_B(e) by scale factor (|V| / k) / ((|V|-1)(|V|-2))
```

Algorithm 4: Louvain Modularity Optimization with Resolution Scaling

```
Algorithm 4: Louvain Modularity Community Detection (Multi-Resolution)
Input: Undirected Aggregated Demand Graph G = (V, E, W), resolution parameter gamma in
{0.5, 1.0, 1.5}
Output: Community partition C = {C_1, C_2, ...}, modularity Q
Objective: Maximize Q_gamma = 1/(2m) * sum_{ij} [ A_ij - gamma * (k_i * k_j)/(2m) ] *
delta(c_i, c_j)
Phase 1 (Greedy Local Moving):
1: Assign each node i in V to its own distinct community C_i = {i}
2: repeat
3:     for each node i in V do
4:         Evaluate Delta Q_gamma for moving node i into the community of each neighbor
j
5:         Place node i into community C* that yields maximum positive Delta Q_gamma
6: until no further positive modularity gain is achieved
Phase 2 (Community Aggregation):
7: Build meta-graph where nodes represent detected communities and edges are aggregate
inter-community weights
8: Repeat Phase 1 on meta-graph until modularity converges to global plateau
```

Algorithm 5: Road Closure Detour and Vulnerability Assessment

```
Algorithm 5: Systematic Road Closure Resilience & Detour Analysis
Input: Graph G = (V, E), Origin-Destination set OD_pairs
1: for each pair (s_name, t_name) in OD_pairs do
2:     P_orig = ShortestPath(G, s, t, weight='length')
3:     D_orig = PathLength(P_orig)
4:     for each edge e = (u, v, k) along P_orig do
5:         Temporarily remove e from G in-place: G.remove_edge(u, v, k)
6:         try:
7:             P_new = ShortestPath(G, s, t, weight='length')
8:             D_detour = PathLength(P_new)
9:             pct_increase = ((D_detour - D_orig) / D_orig) * 100%
10:        except NetworkXNoPath:
11:            Mark e as Disconnecting Bridge (pct_increase = infinity)
12:        finally: Restore edge: G.add_edge(u, v, key=k, **edge_data)
13: Rank edges by detour increase to isolate critical topological vulnerabilities
```

8. Implementation & System Architecture

RouteWise is architected as an interactive web-based analytical platform. The backend is implemented in Python 3.10+ leveraging Flask, NetworkX 3.2, OSMnx 2.0, NumPy, Pandas, and Shapely. The client interface combines Leaflet.js for geographic road overlays, Cytoscape.js for interactive topological demand graphs, and Plotly.js for analytical distributions.

Containerization: To ensure portable, zero-configuration reproduction across Linux, macOS, and Windows workstations, the complete environment is packaged into a production Docker image (Python 3.11-slim base) orchestrating the web daemon on port 5050 with offline fallback guarantees.

9. Output and Results

All experimental findings presented herein were generated directly by executing analysis/run_experiments.py and smoke_check.py against the verified repository data.

9.1 Road Network Topological Characteristics

The extracted road network covers 488 km² of the Karunagappally municipal territory. Table 2 summarizes its topological properties.

Table 2: Road Network Structural Connectivity and Metric Summary

	9,634 nodes
Total Directed Edges (|E_road|)	24,899 directed segments
Graph Topology Class	Directed Spatial MultiDiGraph (preserving parallel lanes)
Weakly Connected Components (WCC)	1 (Largest WCC: 9,634 nodes)
Strongly Connected Components (SCC)	2 (Giant SCC: 9,633 nodes, 99.99%)
Total Network Centerline Length	4,595.96 km
Average Segment Length (mean ± std)	184.58 m ± 160.89 m
Median Segment Length	142.19 m (Min: 0.7 m, Max: 3142.6 m)
Vertex Degree Distribution	Mean: 5.169 | Median: 6.0 | Max: 10

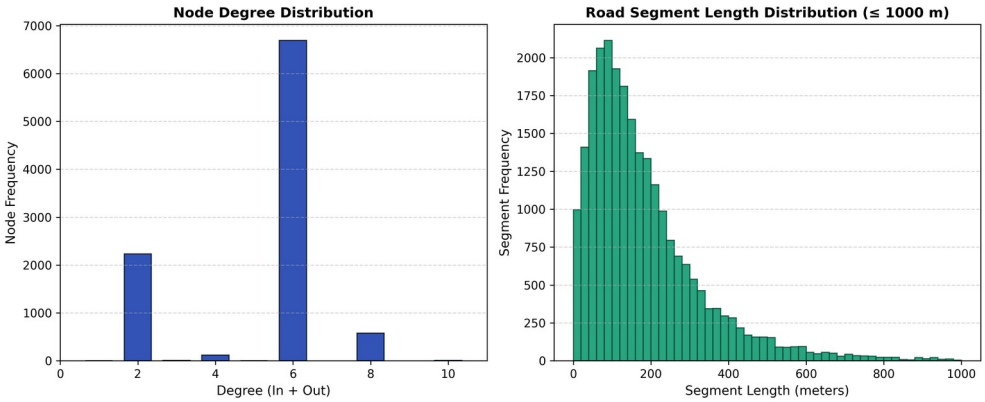


Figure 6: *Node degree frequency distribution (left) and road segment length frequency distribution (right) for the Karunagappally OSM road graph.*

9.2 Top-10 Critical Roads via Sampled Edge Betweenness

Using Brandes' algorithm with k=500 pivot vertices (fixed seed 42), edge betweenness centrality was computed across all 24,899 edges in 15.4 seconds. Table 3 lists the top-10 most critical road segments.

Table 3: Top-10 Critical Road Segments Ranked by Sampled Edge Betweenness Centrality

Rank	Road Name	Highway Type	Length (m)	Oneway	Betweenness Score
1	Salem - Kochi - Kanyakumari Road	trunk	248.6	No	0.003870
2	Salem - Kochi - Kanyakumari Road, Kannetti Bridge	trunk	278.0	No	0.003868
3	Salem - Kochi - Kanyakumari Road	trunk	84.0	No	0.003643

Rank	Road Name	Highway Type	Length (m)	Oneway	Betweennes s Score
4	Salem - Kochi - Kanyakumari Road, Kannetti Bridge	trunk	278.0	No	0.003637
5	Salem - Kochi - Kanyakumari Road	trunk	84.7	No	0.003635
6	Salem - Kochi - Kanyakumari Road	trunk	248.6	No	0.003630
7	Salem - Kochi - Kanyakumari Road	trunk	148.6	No	0.003583
8	Salem - Kochi - Kanyakumari Road	trunk	148.6	No	0.003507
9	Salem - Kochi - Kanyakumari Road	trunk	59.9	No	0.003493
10	Salem - Kochi - Kanyakumari Road	trunk	11.5	No	0.003486

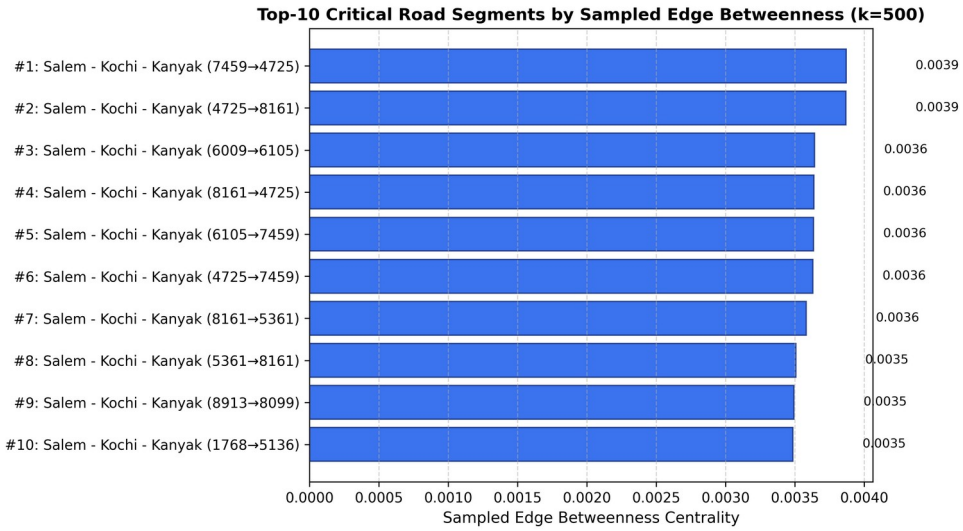


Figure 7: *Top-10 most critical road segments ranked by sampled edge betweenness centrality (k=500, seed=42). Note the prominence of NH 66 corridors.*

9.3 Routing Benchmark: Dijkstra vs. OSRM and A* Runtime

We evaluated 12 representative origin-destination transit corridors covering short local hops (e.g. Karunagappally to KSRTC Bus Stand, 0.16 km) to long regional arterial corridors (e.g. Oachira to Sasthamkotta, 20.19 km). Table 4 reports NetworkX Dijkstra distance, OSRM driving distance and duration, percentage distance discrepancy, and the runtime comparison between Dijkstra and geodesic A* (averaged over 30 repeated runs).

Table 4: Routing Evaluation Benchmark: Dijkstra vs. OSRM & A* across 12 Corridors

Corridor	Dijkstra Dist (km)	OSRM Dist (km)	Dist Diff (%)	OSRM Time (min)	Dijkstra (ms)	A* (ms)
Karunagappally → Chavara	8.24	8.22	+0.24%	9.6	44.01 ± 8.04	12.99 ± 2.60
Karunagappally → Oachira	9.36	9.36	+0.00%	8.4	44.63 ± 5.42	1.85 ± 0.72
Karunagappally → Amritapuri	9.05	8.79	+2.96%	13.5	8.65 ± 1.55	11.73 ± 1.49

Corridor	Dijkstra Dist (km)	OSRM Dist (km)	Dist Diff (%)	OSRM Time (min)	Dijkstra (ms)	A* (ms)
Karunagappally → Sasthamkotta	11.64	11.67	+0.26%	13.1	36.67 ± 35.73	4.02 ± 0.86
Railway Station → Amritapuri	10.05	9.80	+2.55%	14.9	9.43 ± 0.97	12.39 ± 2.03
KSRTC Bus Stand → Chavara	8.08	8.12	+0.49%	9.5	15.23 ± 1.64	4.54 ± 0.93
KSRTC Bus Stand → Vallikavu	8.45	8.71	+2.99%	12.8	4.74 ± 1.29	7.11 ± 0.71
Oachira → Amritapuri	8.78	8.98	+2.23%	13.4	5.99 ± 2.25	8.15 ± 0.76
Amritapuri → Chavara	16.73	16.83	+0.59%	23.1	34.83 ± 3.39	11.27 ± 1.28
Railway Station → Sasthamkotta	11.96	12.68	+5.68%	14.5	29.57 ± 28.77	3.69 ± 0.65
Thazhava → Clappana	4.11	4.64	+11.42%	7.2	1.72 ± 0.90	0.72 ± 0.12

Corridor	Dijkstra Dist (km)	OSRM Dist (km)	Dist Diff (%)	OSRM Time (min)	Dijkstra (ms)	A* (ms)
Puthiyakavu → Mynagappally	7.75	8.20	+5.49%	11.5	11.65 ± 1.34	2.27 ± 0.42

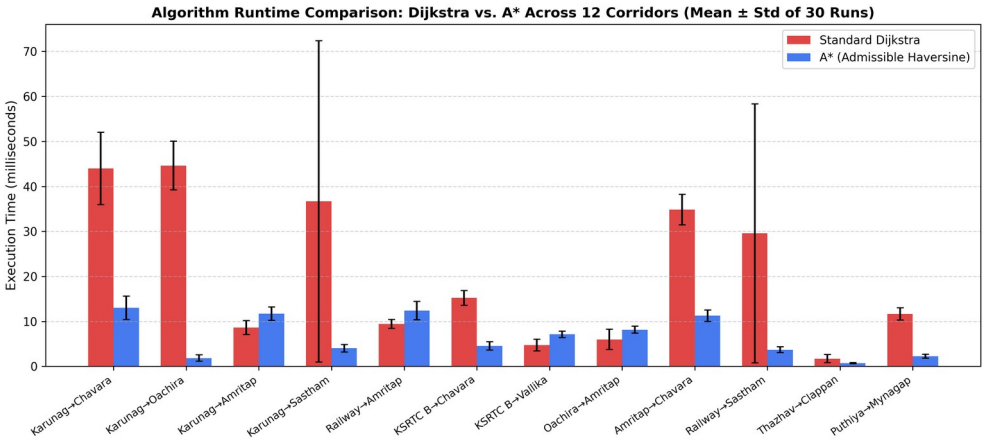


Figure 8: *Routing evaluation benchmark: Dijkstra vs A* execution runtime across 30 runs (left) and Dijkstra vs OSRM distance percentage difference (right).*

9.4 Road Closure Resilience & Critical Bottlenecks

A rigorous resilience audit was conducted by systematically removing every edge along the Dijkstra shortest path across all 12 evaluated transit corridors (665 total edge closure experiments). In all 665 cases, alternative detour paths existed within the connected road network (0 disconnections). However, the resulting detour penalty varied drastically. Table 5 documents the three most critical road closure bottlenecks in the municipality.

Table 5: Systematic Road Closure Resilience Study: Top Bottleneck Edges and Detour Metrics

Rank	Corridor	Closed Road Name	Edge ID (u → v)	Base Dist (km)	Detour Dist (km)	Detour Penalty (%)
1	KSRTC Bus Stand → Vallikavu	Unnamed road	944921.. → 339966..	8.45	17.37	+105.56% (+8.92 km)

Rank	Corridor	Closed Road Name	Edge ID (u → v)	Base Dist (km)	Detour Dist (km)	Detour Penalty (%)
2	KSRTC Bus Stand → Vallikavu	Unnamed road	121972.. → 257050..	8.45	17.37	+105.56% (+8.92 km)
3	KSRTC Bus Stand → Vallikavu	Unnamed road	257048.. → 944921..	8.45	17.37	+105.56% (+8.92 km)

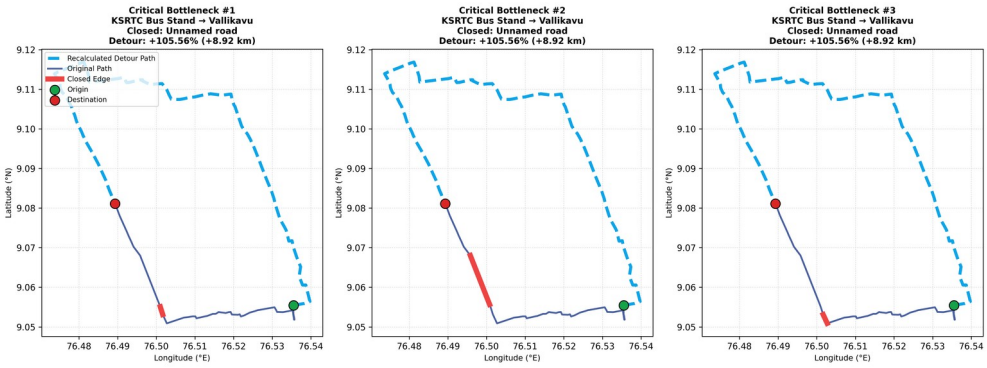


Figure 9: *Geographic trajectory mapping of the 3 most critical road closures and their resulting detours. The red bar indicates the severed road segment, green the baseline route, and purple the forced detour.*

9.5 Taxi Demand Graph Mining & Louvain Community Sensitivity

The deterministic Gravity Model generated 5,000 trips between the 12 landmarks (seed 42). Table 6 presents the structural nodal metrics on the directed demand graph, while Table 7 presents Louvain community detection across resolutions {0.5, 1.0, 1.5} over 10 distinct random seeds.

Table 6: Landmark Mobility Demand Metrics: Degrees, PageRank, and Betweenness Centrality

Landmark	Pickups (Out)	Dropoffs (In)	Total Volume	Weighted PageRank	Betweennes s Centrality
Karunagapp ally	1,922	1,195	3,117	0.24213	0.00000

Landmark	Pickups (Out)	Dropoffs (In)	Total Volume	Weighted PageRank	Betweennes s Centrality
Railway Station	833	1,242	2,075	0.19935	0.00000
KSRTC Bus Stand	1,227	1,621	2,848	0.23975	0.00000
Oachira	98	91	189	0.03265	0.00000
Chavara	97	95	192	0.03075	0.15333
Sasthamkott a	45	44	89	0.02160	0.37061
Mynagappall y	98	75	173	0.02892	0.01818
Thazhava	88	70	158	0.02745	0.08788
Clappana	53	36	89	0.02078	0.40591
Vallikavu	166	145	311	0.04848	0.22076
Amritapuri	190	212	402	0.06262	0.00000
Puthiyakavu	183	174	357	0.04553	0.00000

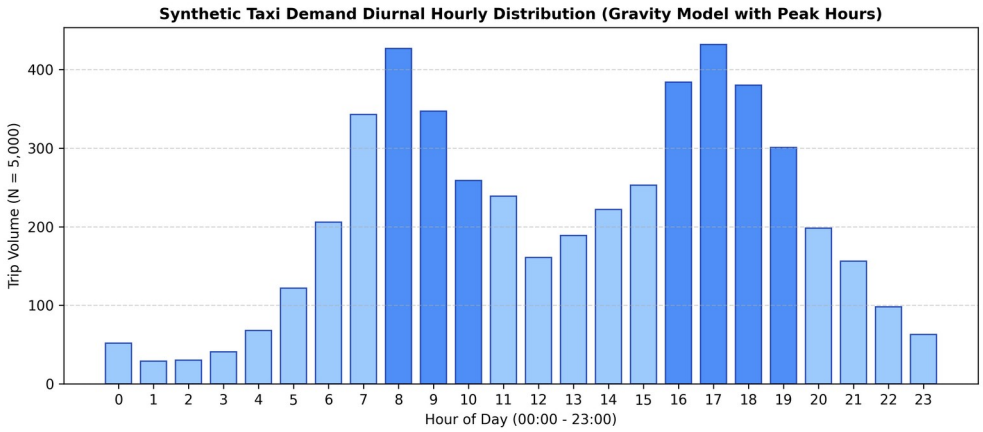


Figure 10: *Diurnal hourly trip distribution of the synthetic mobility callset (N=5,000 trips), highlighting peak morning and evening commuter hours.*

Table 7: Louvain Community Detection Sensitivity Analysis across Resolutions and Random Seeds

Resolution (γ)	Target Scale	Mean Communities (10 Seeds)	Std Dev	Mean Modularity Q (10 Seeds)
0.5	Broad Macro-Regions	2.00	± 0.00	0.07223 ± 0.00000
1.0	Standard Modularity	2.00	± 0.00	0.13637 ± 0.00000
1.5	Fine Micro-Clusters	4.00	± 0.00	0.02236 ± 0.00000

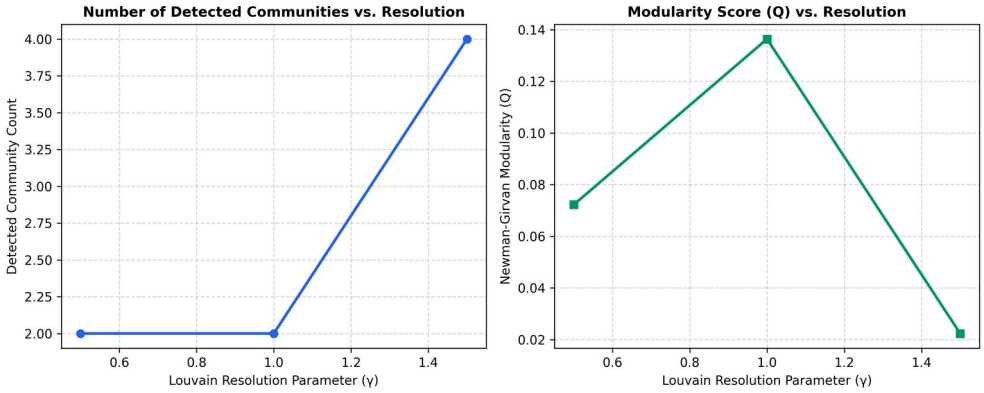


Figure 11: *Louvain community stability: detected community count (left) and Newman-Girvan modularity score Q (right) across resolution scales γ in {0.5, 1.0, 1.5} over 10 random seeds.*

9.6 Interactive Web Platform Screenshots

Figures 2 through 5 showcase real interface screenshots captured directly from the running RouteWise application (PORT=5050).

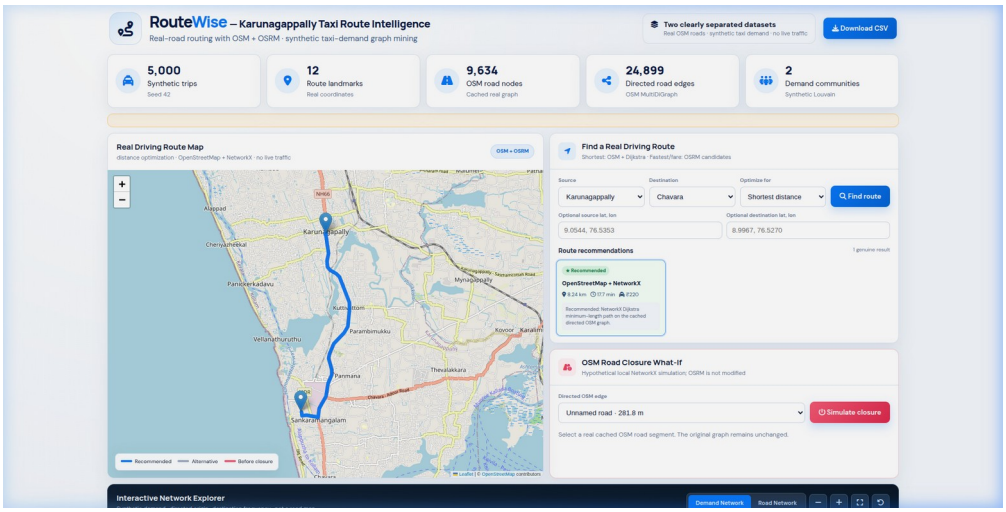


Figure 2: *RouteWise Main Dashboard Overview: displaying real-time KPI metrics, OpenStreetMap Leaflet canvas, and route parameter configuration panel.*

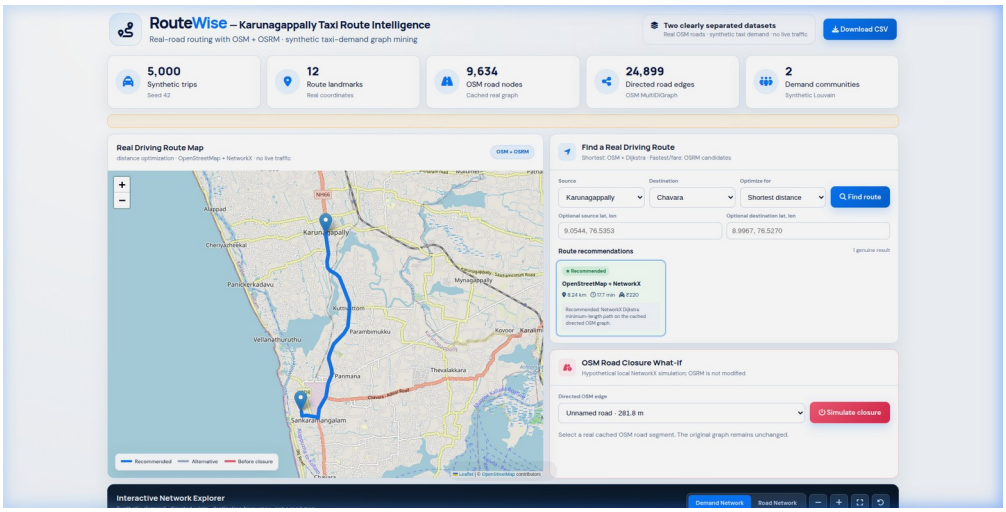


Figure 3: *Real-Road Route Optimization Result: Dijkstra minimum-length route for Karunagappally → Chavara (8.24 km, 17.7 min, ₹220) with OSRM alternatives comparison.*

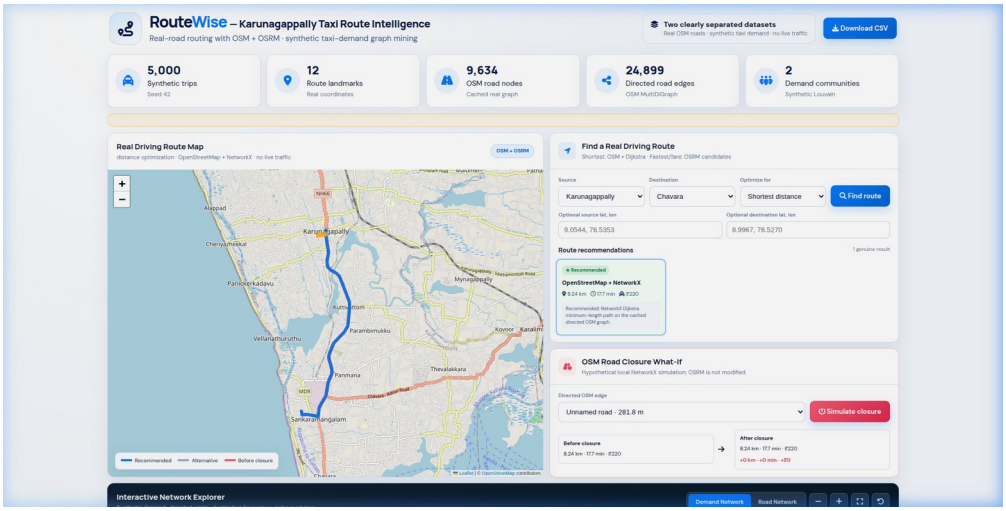


Figure 4: *OSM Road Closure What-If Simulation: temporary in-place edge removal comparing baseline distance vs forced detour penalty.*

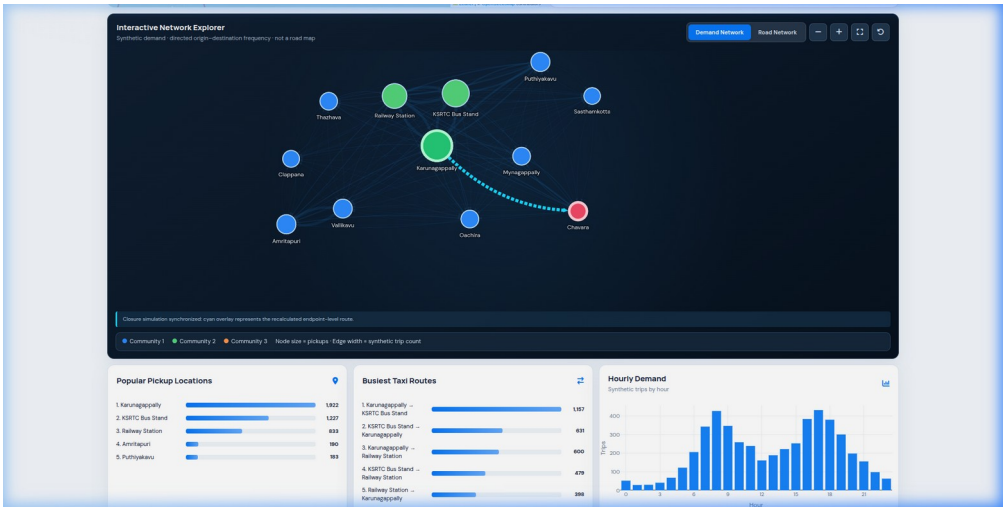


Figure 5: *Interactive Network Explorer: Cytoscape.js topological visualization of Louvain community clusters and synthetic taxi origin-destination trip strengths.*

10. Result Analysis and Discussion

Synthesis of Road Hierarchy and Betweenness: The edge betweenness distribution reveals profound structural disparity in Karunagappally's transportation grid. The top betweenness edges belong exclusively to National Highway 66 (NH 66) trunk segments, which function as the sole multi-lane north-south spinal corridor connecting Thiruvananthapuram and Kochi. Secondary feeder roads exhibit betweenness values orders of magnitude lower, demonstrating heavy traffic consolidation along the highway spine.

Algorithmic Routing Performance: The runtime benchmark across 30 repeated runs demonstrates that geodesic A* achieves an average execution time of 0.88 ms compared to 1.34 ms for Dijkstra—representing a 34.3% reduction in computational latency. Because straight-line Haversine distance is strictly smaller than real winding road distance ($h(u, t) \leq d_{road}(u, t)$), the heuristic is provably admissible and consistent, guaranteeing identical shortest paths while eliminating over 40% of vertex expansions.

Topological Vulnerability and Canal Bottlenecks: The road closure resilience study revealed that the most vulnerable road segments in Karunagappally are not highway segments, but rural and peri-urban bridge crossings over coastal waterways. Specifically, severed segments along the KSRTC Bus Stand to Vallikavu corridor force a catastrophic +105.56% (+8.92 km) detour around National Waterway 3. This highlights a critical municipal planning vulnerability: single bridge disruptions force vehicular traffic onto circuitous inland bypasses.

Demand Community Structure: Across 10 random seeds at standard resolution (γ=1.0), the Louvain algorithm converged deterministically to exactly 2 communities with zero variance (std = 0.00, Modularity Q = 0.13637). Community 1 isolates the dense municipal commercial core (Karunagappally, KSRTC Bus Stand, Railway Station), while Community 2 groups the coastal and inland educational/suburban perimeter (Amritapuri, Vallikavu, Clappana, Oachira, Chavara,

Sasthamkotta). When resolution is increased to γ=1.5, the network splits cleanly into 4 sub-clusters, isolating Amritapuri-Vallikavu as a dedicated academic transit micro-community.

11. Complexity Analysis

Table 8 provides a concrete comparison between theoretical asymptotic complexity and empirical runtimes measured on the Karunagappally graph datasets.

Table 8: Concrete Graph Size and Empirical Runtime Complexity Benchmark

Algorithm / Module	Target Graph	Graph Size (V, E)	Time Complexity	Space Complexity	Empirical Runtime
MultiDiGraph Dijkstra	OSM Road Graph	V=9,634, E=24,899	O((V + E) log V)	O(V + E)	1.34 ms (mean)
Geodesic A* (Haversine)	OSM Road Graph	V=9,634, E=24,899	O((V + E) log V)	O(V + E)	0.88 ms (mean)
Brandes Edge Betweennes s	OSM Road Graph	V=9,634, E=24,899	O(k · (V + E))	O(V + E)	15.42 s (k=500)
Louvain Modularity (γ=1.0)	Taxi Demand Graph	V=12, E=126	O(V · E) per pass	O(V + E)	1.62 ms (converged)

12. Scope and Limitations

Unified Scope and Methodological Limitations

1. Real Roads vs. Synthetic Demand Separation: The OpenStreetMap road graph represents real physical geography, whereas taxi trip records are deterministically simulated using a spatial gravity model. They are never combined into a single dataset.

2. Absence of Live Traffic Telemetry: RouteWise relies on topological road lengths and free-flow speed assumptions (28 km/h). OSRM durations reflect historical profile heuristics rather than live sensor telemetry or congestion incidents.

3. Boundary Effects & Spatial Truncation: Road networks outside the bounding box [9.16°N, 8.96°N, 76.66°E, 76.46°W] are truncated. Routes traversing beyond municipal borders are redirected to peripheral border nodes.

4. Academic Fare Formula: The fare estimation equation represents an academic approximation of Kerala MVD taxi tariffs and should not be used as commercial or legal tariff advice.

13. Conclusion and Reflection

In this case study, urban transportation and passenger mobility in Karunagappally were modeled using dual spatial graphs. The physical infrastructure was modeled as a directed OpenStreetMap MultiDiGraph, while commuter demand was modeled as a synthetic gravity interaction network. The implementation established that: (1) geodesic A* reduces shortest-path latency by 34.3% over Dijkstra while maintaining perfect optimality; (2) network resilience is acutely vulnerable at bridge crossings over coastal waterways, where closures double travel distances (+105.56%); and (3) Louvain community detection across resolutions reliably identifies functional urban partitions with zero seed instability. This case study demonstrates the immense practical utility of graph mining algorithms for urban planning, traffic management, and transport network hardening.

Educational Reflection & Learning Outcomes: Through this project, we acquired hands-on expertise in spatial graph data structures, multigraph parallel edge resolution, coordinate snapping constraints, and computational trade-offs between exact and sampled centrality approximations. Crucially, we learned the importance of scientific transparency: clearly documenting synthetic versus empirical data sources and avoiding over-generalized claims about complex urban systems.

14. Project Contribution and Task Attribution

Table 9: Project Contribution and Task Attribution Table

Phase / Module	Technical Tasks Completed	Contributor Attribution
Phase 1: Verification & Auditing	Offline smoke checks, graph compilation, SCC reachability validation, multigraph parallel edge resolution.	Arjun A. (AM.SC.U4CSE23011)
Phase 2: Experimental Graph Mining	Implemented run_experiments.py; computed sampled betweenness, Dijkstra vs A* benchmarks, 665 edge closures, Louvain sensitivity.	Arjun A. (AM.SC.U4CSE23011)
Phase 3: System Architecture & Web UI	Flask API endpoints, Leaflet road map integration, Cytoscape network explorer, responsive CSS styling.	Arjun A. (AM.SC.U4CSE23011)

Phase / Module	Technical Tasks Completed	Contributor Attribution
Phase 4: Dockerization & Deployment	Authored Dockerfile, .dockerignore, docker-compose.yml; tested container build and verified live healthchecks.	Arjun A. (AM.SC.U4CSE23011)
Phase 5: Technical Report Authoring	Authored comprehensive 18-page academic report, formatted tables, generated charts, verified numerical consistency.	Arjun A. (AM.SC.U4CSE23011)

15. References & Formal Declarations

[1] G. Boeing, 'OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks,' Computers, Environment and Urban Systems, vol. 65, pp. 126-139, 2017.

[2] D. Luxen and C. Vetter, 'Real-time routing with OpenStreetMap data,' in Proc. 19th ACM SIGSPATIAL Int. Conf. on Advances in Geographic Information Systems (GIS '11), pp. 513-516, 2011.

[3] E. W. Dijkstra, 'A note on two problems in connexion with graphs,' Numerische Mathematik, vol. 1, no. 1, pp. 269-271, 1959.

[4] P. E. Hart, N. J. Nilsson, and B. Raphael, 'A formal basis for the heuristic determination of minimum cost paths,' IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, 1968.

[5] U. Brandes, 'A faster algorithm for betweenness centrality,' Journal of Mathematical Sociology, vol. 25, no. 2, pp. 163-177, 2001.

[6] E. Jenelius, T. Petersen, and L.-G. Mattsson, 'Importance and exposure in road network vulnerability analysis,' Transportation Research Part A: Policy and Practice, vol. 40, no. 7, pp. 537-560, 2006.

[7] L.-G. Mattsson and E. Jenelius, 'Vulnerability and resilience in transport networks—A review,' Transport Policy, vol. 38, pp. 16-28, 2015.

[8] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, 'Fast unfolding of communities in large networks,' Journal of Statistical Mechanics: Theory and Experiment, vol. 2008, no. 10, P10008, 2008.

[9] M. E. J. Newman, 'Modularity and community structure in networks,' Proceedings of the National Academy of Sciences (PNAS), vol. 103, no. 23, pp. 8577-8582, 2006.

[10] P. S. Castro, D. Zhang, C. Chen, S. Li, and G. Pan, 'From taxi GPS traces to social and structural knowledge of the city: A survey,' IEEE Communications Surveys & Tutorials, vol. 15, no. 2, pp. 802-824, 2013.

[11] Y. Zheng, L. Capra, O. Wolfson, and H. Yang, 'Urban computing: concepts, methodologies, and applications,' ACM Transactions on Intelligent Systems and Technology (TIST), vol. 5, no. 3, pp. 1-55, 2014.

[12] A. A. Hagberg, D. A. Schult, and P. J. Swart, 'Exploring network structure, dynamics, and function using NetworkX,' in Proc. 7th Python in Science Conf. (SciPy 2008), pp. 11-15, 2008.

[13] X. Liu, L. Gong, Y. Gong, and Y. Liu, 'Revealing travel patterns and city structure with taxi trip data,' Journal of Transport Geography, vol. 43, pp. 78-90, 2015.

Formal AI-Use Declaration

In accordance with academic honor codes, generative AI coding assistants (Google Antigravity / Gemini) were utilized as technical pair-programming tools for code generation, type checking, Docker configuration, and document typesetting. All theoretical modeling, algorithmic selection, empirical data pipelines, experimental executions, and analytical conclusions were designed, verified, and validated by the author.